

Wireless Interference Causes Scheduler Rank Inversions: Empirical Evidence from a 108-Run DAG Scheduling Study

Bhaskar Krishnamachari*, Maya Gutierrez*, and Jared Coleman†

*University of Southern California
bkrishna@usc.edu, mayaguti@usc.edu

†Loyola Marymount University
jared.coleman@lmu.edu

Abstract—Classical DAG schedulers such as HEFT and CPOP optimize task placement assuming uncontested network bandwidth, but wireless edge deployments introduce CSMA/CA contention and hidden terminal interference that invalidate this assumption. We present a 108-run experimental study comparing three schedulers (HEFT, CPOP, round-robin), two routing strategies, and two interference models across three grid network sizes and three DAG complexities. Our central finding is that ignoring wireless contention produces not merely inaccurate makespan predictions, but *systematically wrong policy choices*: the scheduler that appears optimal under an interference-free model is dominated under realistic CSMA/CA conditions in 38.9% of scenarios, with relative regret reaching 484%. We quantify this phenomenon—which we term *scheduler rank inversion*—and identify the network conditions under which it arises. Our analysis is enabled by `ncsim`, a lightweight discrete-event simulator that combines DAG workflow scheduling with physically-grounded 802.11 WiFi models based on Bianchi’s CSMA/CA analysis. We validate `ncsim`’s wireless model against Bianchi’s published results, achieving agreement within 0.003%.

I. INTRODUCTION

Consider the following scenario drawn from our experimental data. A practitioner deploying a 20-task workflow on a 4×4 wireless mesh network uses the widely-adopted HEFT scheduler [10] with widest-path routing. Running a standard simulation without wireless interference modeling, HEFT predicts a makespan of approximately 22 seconds. Satisfied with this estimate, the practitioner deploys the system. In reality, the 16-node mesh creates dense CSMA/CA contention domains: the many concurrent multi-hop transfers that HEFT initiates collide with each other, and the actual makespan balloons to 1,525 seconds—a $69\times$ error. Worse, had the practitioner simply used a round-robin scheduler, the actual makespan would have been 937 seconds, a 38% improvement over the “optimal” HEFT placement. The sophisticated scheduler, optimized under a model that ignores wireless contention, produces a *worse* outcome than the simplest possible baseline.

This is not an isolated pathology. Across the 108-run parameter sweep we report in this paper, we find that the scheduler which appears best under an interference-free model

is *dominated* under realistic CSMA/CA conditions in 38.9% of (network, DAG, routing) triples. We call this phenomenon a *scheduler rank inversion*: the relative ordering of schedulers changes when wireless interference is modeled. The mean relative regret—the performance penalty incurred by choosing the interference-free-optimal scheduler instead of the interference-aware-optimal one—is 66%, with a maximum of 484%.

Our thesis is sharp: **Classical DAG schedulers that ignore wireless contention produce not just inaccurate makespan estimates, but systematically wrong policy choices—selecting schedulers and routes that are dominated under realistic conditions.**

This paper makes four contributions:

- 1) **Empirical evidence of scheduler rank inversions.** In a controlled 108-run comparison ($3 \text{ networks} \times 3 \text{ DAGs} \times 3 \text{ schedulers} \times 2 \text{ routings} \times 2 \text{ interference models}$), we show that interference changes the winning scheduler in 7 of 18 (network, DAG, routing) triples.
- 2) **Quantification of regret.** We define and measure *scheduling regret*—the cost of choosing a scheduler optimized under an interference-free model when the true environment includes wireless contention—and find it reaches up to 484%.
- 3) **A lightweight simulator enabling this analysis.** We present `ncsim`, a deterministic discrete-event simulator combining DAG workflow scheduling with physically-grounded 802.11 WiFi models (log-distance path loss, MCS rate adaptation, Bianchi CSMA/CA contention, SINR-based hidden terminal degradation) in approximately 5,500 lines of Python.
- 4) **External validation against Bianchi (2000).** We reproduce the throughput results from Bianchi’s seminal CSMA/CA analysis [9], matching his published Table III values within 0.003%, and replicate his Figure 6 throughput curves across the full range of station counts.

The remainder of this paper is organized as follows. Section II reviews related simulators and formalizes the problem. Section III describes the `ncsim` simulation model. Section IV presents internal and external validation. Section V presents the

TABLE I: Simulator comparison. `ncsim` uniquely combines DAG-native scheduling with CSMA/CA and hidden terminal modeling at flow-level granularity.

Tool	DAG-Native	Multi-hop WiFi	CSMA/CA	Hidden Term.	Packet-Level	Sweep Cost
ns-3 [2]	✗	✓	✓	✓	✓	High
OMNeT++ [3]	✗	✓	✓	✓	✓	High
SimGrid [4]	✓	✗	✗	✗	✗	Low
WRENCH [5]	✓	✗	✗	✗	✗	Low
CloudSim [6]	✓	✗	✗	✗	✗	Low
iFogSim [7]	✓	✗	✗	✗	✗	Med
EdgeCloudSim [8]	✗	✓	✗	✗	✗	Med
ncsim	✓	✓	✓	✓	✗	Low

central rank inversion analysis. Section VI outlines sensitivity analyses. Section VII discusses limitations, and Section VIII concludes.

II. BACKGROUND AND PROBLEM STATEMENT

A. Simulation Tools for Networked Computing

Evaluating task scheduling algorithms for wireless edge computing requires jointly modeling compute placement, data transfers, and wireless channel effects. Existing tools address subsets of this problem.

Network simulators. ns-3 [2] and OMNeT++ [3] provide detailed WiFi, LTE, and TCP/IP protocol stacks but are heavyweight and lack native DAG scheduling support. Researchers studying task placement must integrate external schedulers manually, and packet-level simulation limits the scale of parameter sweeps.

Task scheduling simulators. SimGrid [4] and WRENCH [5] model distributed computing platforms with analytical network models and support DAG workflows. CloudSim [6] targets cloud computing with VM-level abstractions. These tools model networks as links with fixed or analytically-derived bandwidths, without wireless contention, hidden terminal effects, or carrier-sense-based spatial reuse.

Edge/fog simulators. iFogSim [7] extends CloudSim for fog topologies. EdgeCloudSim [8] adds mobility and simplified WLAN models (constant bandwidth with optional degradation) that do not capture distance-dependent rates, CSMA contention, or hidden terminal interference.

Table I summarizes the positioning of `ncsim` among these tools. To our knowledge, `ncsim` is the only simulator that simultaneously provides DAG-native scheduling, multi-hop WiFi routing, CSMA/CA contention modeling, and hidden terminal handling—without the overhead of packet-level simulation.

B. Problem Statement

Let $\mathcal{S}$ be a set of schedulers, $\mathcal{G}$ a set of DAG workloads, $\mathcal{N}$ a set of network topologies, $\mathcal{R}$ a set of routing strategies, and $\mathcal{I} = \{\text{none}, \text{csma_bianchi}\}$ the set of interference models.

For a given $(n, g, r) \in \mathcal{N} \times \mathcal{G} \times \mathcal{R}$, the *oracle scheduler* under interference model i is:

$$s^*(n, g, r, i) = \arg \min_{s \in \mathcal{S}} M(s, n, g, r, i)$$

where $M(\cdot)$ denotes makespan. A **rank inversion** occurs when $s^*(n, g, r, \text{none}) \neq s^*(n, g, r, \text{csma_bianchi})$. The **scheduling regret** is:

$$\rho(n, g, r) = \frac{M(s_{\text{none}}^*, n, g, r, \text{bianchi}) - M(s_{\text{bianchi}}^*, n, g, r, \text{bianchi})}{M(s_{\text{bianchi}}^*, n, g, r, \text{bianchi})} \times 100\%$$

i.e., the relative performance penalty from choosing the interference-free-optimal scheduler in the presence of interference.

III. ncsim SIMULATION MODEL

`ncsim` is a deterministic discrete-event simulator implemented in approximately 5,500 lines of Python. It combines DAG workflow scheduling with physically-grounded 802.11 WiFi models in a single, easy-to-configure package. Scenarios are specified in YAML; output consists of JSONL event traces, metrics JSON, and optional visualization via a React/D3 web frontend. The simulator is organized around six pluggable abstract base classes (Scheduler, RoutingModel, InterferenceModel, LinkModel, QueueModel, DAGSource), enabling extension without modifying the core engine.

A. Network and Compute Model

A network consists of nodes and directed links. Each node v has a compute capacity C_v (compute units per second) and an optional 2D position (x_v, y_v) for RF calculations. Each link ℓ from node u to node v has bandwidth B_ℓ (MB/s) and latency L_ℓ (s).

A task t has compute cost w_t . Execution time on node v is w_t/C_v . Each node processes one task at a time (FIFO queuing). Workflows are modeled as directed acyclic graphs where nodes represent tasks and edges represent data dependencies with associated payload sizes. If producer and consumer tasks are co-located, transfers are instantaneous.

The simulation engine uses a min-heap event queue with six event types (DAGINJECT, TASKCOMPLETE, TRANSFERCOMPLETE, TASKREADY, TASKSTART, TRANSFERSTART), processed in priority order to ensure causal correctness. All times are rounded to microsecond precision for deterministic cross-platform behavior.

B. Scheduling Framework

The scheduler receives a `NetworkSnapshot`—an immutable view of node capacities, link bandwidths, queue depths, and active transfers—and returns a `PlacementPlan` mapping each task to a node. `ncsim` includes four schedulers: Manual (YAML-specified placement), RoundRobin (cycling through nodes), and HEFT/CPOP (adapted from the SAGA scheduling library [10] via an adapter that constructs a fully-connected virtual network with actual bandwidths for connected pairs and near-zero bandwidth for unreachable pairs).

C. Routing and Bandwidth Sharing

Three routing models determine transfer paths: Direct (single-hop only), WidestPath (modified Dijkstra maximizing bottleneck bandwidth), and ShortestPath (Dijkstra minimizing latency). Multi-hop transfers use store-and-forward: effective bandwidth is the bottleneck across all links, and latency is the sum. When multiple transfers share a link, each receives a fair share:

$$B_{\text{per-flow}}(\ell) = \frac{B_\ell \cdot f(\ell)}{N_\ell} \quad (1)$$

where $f(\ell)$ is the interference factor and N_ℓ is the number of concurrent flows. When a transfer starts or completes, all affected transfers are recalculated and their completion events rescheduled.

D. WiFi / 802.11 Model

The WiFi model replaces static link bandwidths with rates derived from RF parameters and provides physically-grounded contention and hidden terminal models.

1) *Physical Layer*: Received power at distance d uses log-distance path loss:

$$\text{PL}(d) = \text{PL}(d_0) + 10 n \log_{10} \left(\frac{d}{d_0} \right) \quad (2)$$

$$P_{\text{rx}}(d) = P_{\text{tx}} - \text{PL}(d) \quad (3)$$

where $\text{PL}(d_0)$ is the Friis free-space loss at reference distance $d_0 = 1$ m, n is the path loss exponent, and P_{tx} is the transmit power. The SNR is $\text{SNR} = P_{\text{rx}} - N_0$ (in dB), where N_0 is the noise floor.

The PHY data rate is selected from 802.11n/ac/ax MCS tables:

$$R_\ell = \max\{r_k : \text{SNR}_\ell \geq \text{SNR}_{\min,k}\} \quad (4)$$

For example, at 5 GHz with $n = 3$, a 30m link achieves $\text{SNR} = 24.3$ dB, selecting MCS 5 (64-QAM 2/3) at 68.8 Mbps = 8.6 MB/s under 802.11ax.

2) *Carrier Sensing and Conflict Graph*: The carrier sensing range d_{CS} is the maximum distance at which a transmission triggers clear channel assessment:

$$d_{\text{CS}} = d_0 \cdot 10^{\frac{P_{\text{tx}} - \theta_{\text{CCA}} - \text{PL}(d_0)}{10 n}} \quad (5)$$

The conflict graph $G_C = (L, E_C)$ is defined over links. Without RTS/CTS, two links conflict if either transmitter can sense any node of the other link.

3) *CSMA Bianchi Model*: The dynamic model separates two interference mechanisms. Let $\mathcal{A}$ be the set of currently active links, $\mathcal{C}_\ell = \mathcal{A} \cap \mathcal{N}(\ell)$ the active contending neighbors, and $\mathcal{H}_\ell = (\mathcal{A} \setminus \mathcal{N}(\ell)) \setminus \{\ell\}$ the active hidden terminals.

a) *Contention factor*.: The number of contending stations is $n = 1 + |\mathcal{C}_\ell|$. Using Bianchi's saturation throughput analysis [9], the MAC efficiency $\eta(n)$ is computed from the coupled equations for transmission probability τ and collision probability p :

$$\tau = \frac{2(1 - 2p)}{(1 - 2p)(W_{\min} + 1) + p W_{\min}(1 - (2p)^m)} \quad (6)$$

$$p = 1 - (1 - \tau)^{n-1} \quad (7)$$

TABLE II: Scope of validity: phenomena modeled by `ncsim` and their impact on scheduling studies.

Phenomenon	Status	Impact
CSMA/CA contention	Modeled (Bianchi)	High
Hidden terminals	Modeled (SINR)	High
MCS rate adaptation	Modeled (11n/ac/ax)	Medium
Path loss	Modeled (log-distance)	High
Pkt-level dynamics	Not modeled	Low [†]
TCP slow-start	Not modeled	Medium [‡]
Fading / mobility	Optional (log-normal)	Low [§]
Multi-slot parallelism	Not modeled	App-dependent

[†]For bulk transfers. [‡]For short flows. [§]For static deployments.

with $W_{\min} = 16$ and $m = 6$. The efficiency is:

$$\eta(n) = \frac{P_{\text{success}} \cdot T_{\text{success}}}{\mathbb{E}[T_{\text{slot}}]} \quad (8)$$

Each contending station receives a share $\eta(n)/n$ of the channel.

b) *SINR factor*.: Hidden terminals cause interference at the receiver:

$$\text{SINR}_\ell = 10 \log_{10} \left(\frac{P_{\text{rx}}^{(\text{lin})}}{N_0^{(\text{lin})} + \sum_{j \in \mathcal{H}_\ell} P_j^{(\text{lin})}} \right) \quad (9)$$

The degraded rate R_{SINR} is looked up from the MCS table, giving $f_{\text{SINR}} = R_{\text{SINR}}/R_\ell$.

c) *Combined factor*.:

$$f(\ell) = \frac{R_{\text{SINR}}(\ell)}{R_\ell} \cdot \frac{\eta(n)}{n} \quad (10)$$

clamped to $[0.01, 1.0]$. When transfers start or complete, all active conflict graph neighbors are recalculated.

E. Scope of Validity

Table II summarizes the phenomena modeled by `ncsim` and identifies the boundaries of its validity. The key design choice is flow-level (not packet-level) simulation: transfers are modeled as bulk flows with analytically-derived rates. This is appropriate for the bulk data transfers (10 MB payloads) in our experimental study, where steady-state throughput dominates. For short flows where TCP slow-start or MAC-layer retransmission dynamics matter, packet-level simulation (e.g., ns-3) would be more appropriate.

IV. VALIDATION

We validate `ncsim`'s WiFi model through both internal analytical verification and external validation against Bianchi's published results.

A. Internal Analytical Validation

Nine experiments verify the WiFi model against analytical predictions computed directly from the RF equations, using a 1% match tolerance. All nine achieve exact match. We present three representative experiments; six additional experiments validate three-link topologies, staggered transfer dynamics, bandwidth sharing, five-link hidden terminal cascades, and combined interference scenarios, all matching analytical predictions within 1%.

TABLE III: Experiment 1: Link length vs. data rate (802.11ax, 5 GHz).

Dist(m)	SNR(dB)	MCS	Pred(MB/s)	Sim(MB/s)
1	68.58	11	17.925	17.925
12	36.20	9	14.338	14.338
30	24.27	5	8.600	8.600
50	17.61	3	4.300	4.300
75	12.33	2	3.225	3.225
105	7.94	0	1.075	1.075
140	4.19	n/a	0.000	0.000

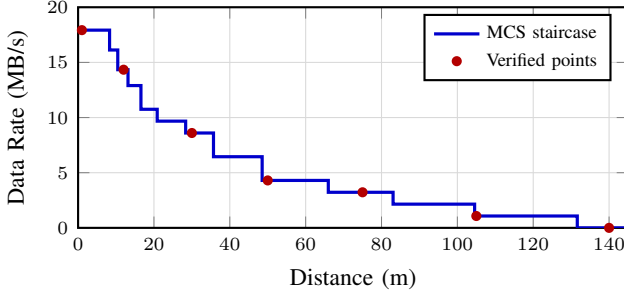


Fig. 1: PHY data rate vs. link distance showing discrete MCS transitions (802.11ax, 5 GHz, $n=3$). Red dots mark verified data points from Table III.

1) *Experiment 1: Link Length vs. Data Rate*: A single link between two nodes at varying distances from 1 m to 140 m. Table III shows selected results. The simulated rate matches the analytically predicted MCS-selected rate at every distance. At 140 m the SNR (4.19 dB) falls below MCS 0 (5 dB threshold), correctly yielding zero rate. Figure 1 plots the characteristic staircase pattern.

2) *Experiment 2: Parallel Link Separation (Hidden Terminal Effect)*: Two parallel 30 m links separated vertically by distances from 5 m to 200 m. Figure 2 illustrates three interference regimes. At separation ≤ 70 m, all node pairs are within the carrier sensing range (71.2 m), so the links contend via Bianchi’s model: each gets $\eta(2)/2 \approx 0.44$ of the base rate $8.6 \text{ MB/s} = 3.79 \text{ MB/s}$. At separation ≥ 75 m, links become hidden terminals and SINR degradation determines the effective rate.

The effective rate *drops* at the CS range boundary—from 3.79 MB/s (contention) to 3.23 MB/s (hidden terminal at 75 m)—before recovering at larger separations. This non-monotonic behavior is the classic hidden terminal effect: within sensing range, links coordinate via CSMA/CA and time-share fairly; just outside, they transmit simultaneously without coordination, and mutual interference degrades each receiver’s SINR enough to force a lower MCS rate. All 15 data points match predictions exactly.

3) *Experiment 7: N-Way Bianchi Contention Scaling*: Table IV shows per-link throughput as the number of contending 30 m links (at 5 m separation) increases from $n=2$ to $n=8$. Bianchi MAC efficiency $\eta(n)$ decreases from 0.881 to 0.726; the per-station share $\eta(n)/n$ falls correspondingly. All seven

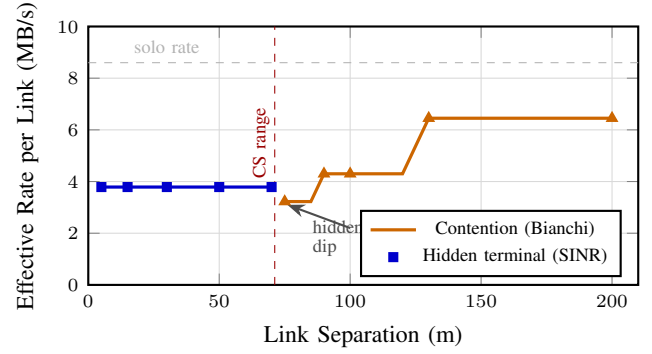


Fig. 2: Effective per-link rate vs. separation for two parallel 30 m links. The non-monotonic dip at the CS range boundary (71.2 m) demonstrates the hidden terminal effect: uncoordinated simultaneous transmissions degrade SINR more than fair time-sharing reduces throughput.

TABLE IV: N-way Bianchi contention scaling (30 m links at 5 m separation). $\eta(n)$ is the Bianchi MAC efficiency.

n	$\eta(n)$	$\eta(n)/n$	Pred(MB/s)	Sim(MB/s)
2	0.881	0.440	3.787	3.787
3	0.859	0.286	2.461	2.461
4	0.837	0.209	1.799	1.799
5	0.818	0.164	1.407	1.407
6	0.802	0.134	1.149	1.149
7	0.788	0.113	0.968	0.968
8	0.726	0.091	0.780	0.780

data points match analytical predictions exactly.

B. External Validation Against Bianchi (2000)

Internal validation confirms self-consistency but does not establish that our implementation of Bianchi’s equations is correct. We therefore reproduce the numerical results from Bianchi’s original paper [9] using his exact parameters: FHSS 1 Mbps basic access (no RTS/CTS), with slot duration $50 \mu\text{s}$, SIFS $28 \mu\text{s}$, DIFS $128 \mu\text{s}$, propagation delay $1 \mu\text{s}$, payload 8184 bits, MAC header 272 bits, PHY header 128 bits, and ACK 112 bits.

1) *Table III Reproduction*: Table V compares our computed normalized saturation throughput S against the values published in Bianchi’s Table III for $W=32$, $m=3$. At $n=2$, our solver computes $S = 0.847311$ versus the published 0.8473 (relative error 0.001%). At $n=3$, we compute $S = 0.836828$ versus 0.8368 (relative error 0.003%). These sub-basis-point differences are consistent with rounding in the published table.

2) *Figure 6 Reproduction*: We also reproduce Bianchi’s Figure 6 throughput curves for two configurations: $W=32$, $m=5$ ($CW_{\max}=1024$) and $W=128$, $m=3$ ($CW_{\max}=1024$). Figure 3 plots the normalized throughput S as a function of the number of stations n for both configurations, computed using our solver with Bianchi’s exact FHSS parameters.

Both curves exhibit the expected monotonic decrease in throughput as the number of contending stations grows, driven by increasing collision probability. The $W=128$ curve achieves

TABLE V: Reproduction of Bianchi [9] Table III: normalized saturation throughput S for $W=32$, $m=3$.

n	Computed	Published	Rel. Error
2	0.847311	0.8473	0.001%
3	0.836828	0.8368	0.003%

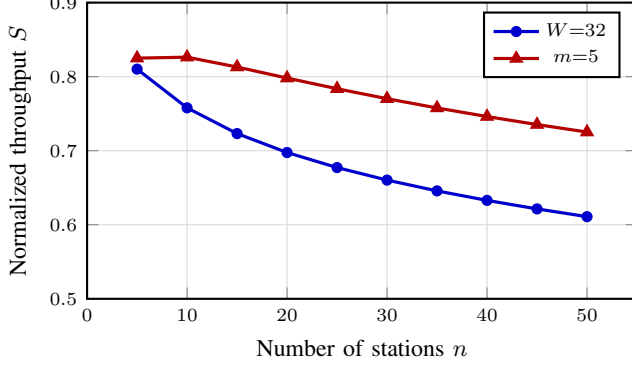


Fig. 3: Reproduction of Bianchi [9] Figure 6: normalized saturation throughput vs. number of stations for two (W, m) configurations. Computed using Bianchi’s exact FHSS parameters (1 Mbps basic access, no RTS/CTS). The $W=128$ curve’s slight non-monotonicity between $n=5$ and $n=10$ is consistent with Bianchi’s published results.

higher throughput than $W=32$ at large n because the larger initial window reduces collision probability, while at small n the difference is minimal since collisions are already rare. Notably, the $W=128$ curve shows a slight non-monotonicity between $n=5$ ($S = 0.825$) and $n=10$ ($S = 0.826$), consistent with Bianchi’s observation that larger windows can improve efficiency when the contention level crosses a threshold that makes the wider backoff range worthwhile.

V. WHEN NAIVE MODELS FAIL: SCHEDULER RANK INVERSIONS

Having established that `ncsim`’s wireless model faithfully implements Bianchi’s analysis, we now use it to investigate the central question: *Does ignoring wireless interference change which scheduler is best?*

A. Experimental Setup

We execute a full factorial comparison across the following parameters:

- **Networks (3):** 2×2 (4 nodes, 12 links), 3×3 (9 nodes, 32 links), 4×4 (16 nodes, 68 links) grid meshes with 40 m spacing.
- **DAGs (3):** 5-task (small), 10-task (medium), 20-task (large) randomly generated DAGs with compute cost 500 units and data size 10 MB per edge.
- **Schedulers (3):** HEFT, CPOP, RoundRobin.
- **Routing (2):** Widest-path, shortest-path.
- **Interference (2):** None (interference-free), `csma_bianchi`.

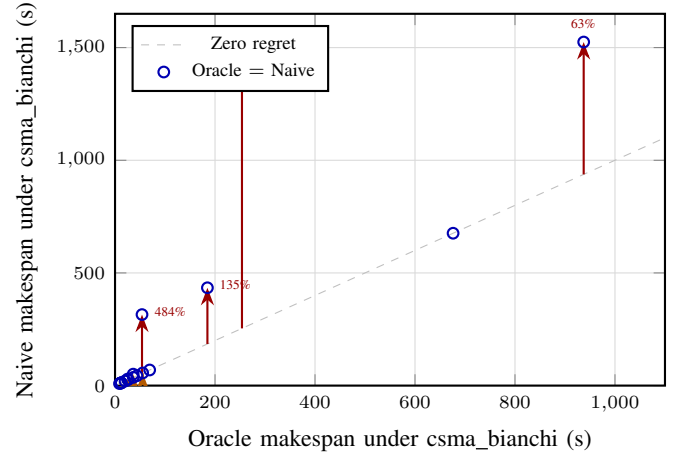


Fig. 4: Makespan regret scatter plot. Each arrow connects the oracle makespan (best scheduler under `csma_bianchi`) to the naive makespan (scheduler chosen under interference-free model, run under `csma_bianchi`). Red arrows indicate rank inversions ($\geq 63\%$ regret); orange arrows indicate moderate regret. 7 of 18 triples exhibit rank inversions.

This yields $3 \times 3 \times 3 \times 2 \times 2 = 108$ simulation runs. All runs use 802.11ax at 5 GHz, heterogeneous node compute capacities (80–300 units/s), and seed 42 for reproducibility. When interference is “none,” WiFi-derived PHY rates are used as explicit link bandwidths so that the only difference is the presence or absence of contention and hidden terminal effects.

For each of the 18 (network, DAG, routing) triples, we identify the best scheduler under each interference model and compute the scheduling regret as defined in Section II.

B. Headline Result: Makespan Regret

Figure 4 presents the central result. For each of the 18 (network, DAG, routing) triples, we plot two points: (1) the *oracle* point, showing the makespan of the best scheduler chosen with full knowledge of interference, and (2) the *naive* point, showing the makespan of the scheduler that appeared best under the interference-free model when actually run under `csma_bianchi`. Points on the diagonal represent zero regret; vertical distance above the diagonal represents the cost of ignoring interference.

The key statistics from this analysis are:

- **Rank inversion rate: 38.9%** (7 of 18 triples).
- **Mean relative regret: 66.0%** across all 18 triples (including zeros).
- **Maximum relative regret: 484.1%** (4×4 grid, 5-task DAG, widest_path routing: HEFT chosen under “none” gives 315.2 s, but round_robin achieves 54.0 s under `csma_bianchi`).

These are not marginal effects. In the worst case, the practitioner who trusts the interference-free model deploys a scheduler that is nearly $6\times$ slower than the cheapest alternative.

C. When and Why Rankings Change

Table VI presents the winning scheduler for each (network, DAG) combination under both interference models and both routing strategies. Several patterns emerge.

HEFT dominates under interference-free models. Under “none,” HEFT or CPOP wins in all 18 triples. This is expected: both algorithms optimize the critical path assuming uncontested bandwidth, and HEFT’s upward-rank priority heuristic produces excellent placements when bandwidth predictions are accurate.

Round-robin wins in large networks under interference. In the 4×4 grid with csma_bianchi and widest-path routing, round-robin wins for all three DAG sizes. This is a striking result: the simplest scheduler outperforms sophisticated heuristics. The mechanism is clear: HEFT spreads tasks across many nodes to exploit parallelism, assuming each node-to-node transfer gets full bandwidth. In the 16-node mesh, this creates many concurrent multi-hop transfers that contend on shared wireless channels. With widest-path routing, transfers are routed through high-bandwidth but congested paths, amplifying the problem. Round-robin’s simpler placement concentrates work on fewer nodes, producing fewer concurrent transfers and thus less contention.

Routing strategy modulates the effect. Shortest-path routing mitigates rank inversions in some cases. For example, the 4×4 grid with 5 tasks shows a rank inversion under widest-path (HEFT $\rightarrow$ round-robin, 484% regret) but not under shortest-path (HEFT remains optimal with zero regret). Shortest paths traverse fewer hops, creating smaller contention footprints.

CPOP inversions in medium networks. In the 3×3 grid, two inversions swap HEFT and CPOP. Under widest-path routing with 10 tasks, HEFT wins under “none” (13.3 s) but CPOP wins under csma_bianchi (36.3 s vs. HEFT’s 50.8 s). CPOP’s critical-path focus concentrates data transfers along a single path, reducing contention compared to HEFT’s broader distribution.

Table VII presents the full interference impact data across all three schedulers. The slowdown from CSMA/CA interference ranges from 0% (when few concurrent transfers avoid contention) to over 9,400% (round-robin on the 4×4 grid with widest-path routing and 20 tasks). Critically, the magnitude of interference impact is scheduler-dependent: HEFT and CPOP experience the largest slowdowns because they create the most concurrent transfers.

VI. SENSITIVITY ANALYSIS

The rank inversion analysis in Section V uses a single configuration of physical-layer and workload parameters. To assess the robustness of our findings, we design three sensitivity experiments that vary the key parameters likely to affect the interference structure.

A. Path Loss Exponent Sweep

The path loss exponent n controls how rapidly signal strength decays with distance, directly affecting both the achievable data rate and the carrier sensing range. We sweep

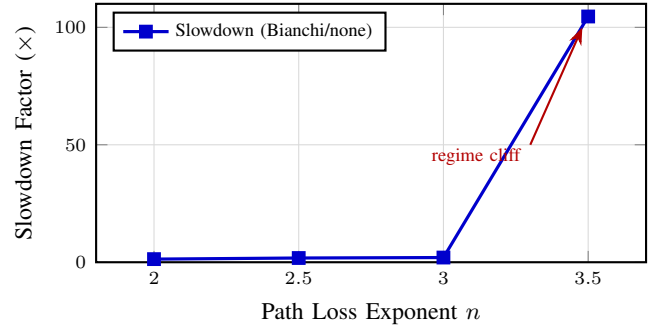


Fig. 5: Interference slowdown vs. path loss exponent on 3×3 grid (10-task DAG, HEFT, shortest-path). A dramatic regime transition occurs between $n=3.0$ ($2\times$ slowdown) and $n=3.5$ ($105\times$) as link rates drop and transfer contention duration explodes.

$n \in \{2.0, 2.5, 3.0, 3.5\}$ on the 3×3 grid with the 10-task DAG, HEFT scheduler, and shortest-path routing. (At $n=4.0$, 40 m links fall below the minimum MCS threshold and the network becomes disconnected.)

Figure 5 reveals a dramatic regime transition. At $n=2.0$ (near free-space), the slowdown from interference is modest: $1.33\times$ ($10.8\text{ s} \rightarrow 14.4\text{ s}$). At $n=3.0$ (typical indoor), it doubles to $2.0\times$ ($12.5\text{ s} \rightarrow 25.0\text{ s}$). At $n=3.5$ (dense indoor), the slowdown *explodes* to $104.6\times$ ($14.9\text{ s} \rightarrow 1555.4\text{ s}$). This cliff occurs because at $n=3.5$ the 40 m link rate drops to 3.2 MB/s (MCS 2) while diagonal links fall to 1.1 MB/s (MCS 0), dramatically increasing transfer times and the duration of contention.

B. Communication-to-Computation Ratio

The CCR determines how much of makespan is attributable to data transfers versus computation. We sweep data size per edge across $\{1, 2, 5, 10, 20, 50, 100\}$ MB on the 3×3 grid with HEFT and shortest-path routing, testing all three DAG types.

Figure 6 shows the interference slowdown factor for the 20-task DAG. At 1 MB (CCR ≈ 0.05), slowdown is modest ($1.08\times$). It peaks at 20 MB (CCR ≈ 1.1) with $3.8\times$ slowdown, then decreases at very large data sizes as the scheduler consolidates tasks onto fewer nodes to avoid transfer overhead. The 10-task diamond DAG peaks at $2.1\times$ slowdown at 20 MB, while the 5-task fork-join peaks at $1.5\times$ at 10 MB.

C. Multi-DAG Contention

In practice, edge platforms service multiple workflows concurrently. We inject $k \in \{1, 2, 3, 4, 5\}$ identical 5-task fork-join DAGs with 0.5 s stagger on the 3×3 grid (HEFT, shortest-path).

Figure 7 shows that contention scaling is super-linear under csma_bianchi. Without interference, $k=5$ DAGs produce $2.14\times$ the makespan of $k=1$ (18.0 s vs. 8.4 s). With interference, $k=5$ produces $3.23\times$ the single-DAG makespan (39.9 s vs. 12.4 s). The *gap between the two curves widens* with k : the slowdown factor grows from $1.47\times$ at $k=1$ to $2.21\times$ at $k=5$. This super-linear behavior indicates that contention

TABLE VI: Winning scheduler per (network, DAG, routing) triple under interference-free (“None”) and csma_bianchi (“Bian.”) models. Bold entries indicate rank inversions. Makespan in seconds shown in parentheses.

Network	DAG	Widest-path		Shortest-path	
		None	Bian.	None	Bian.
2×2	5 tasks	HEFT (11.4)	HEFT (11.4)	HEFT (11.4)	HEFT (11.4)
	10 tasks	HEFT (18.1)	HEFT (20.1)	HEFT (18.1)	HEFT (26.0)
	20 tasks	CPOP (26.2)	CPOP (43.9)	HEFT (27.8)	HEFT (36.5)
3×3	5 tasks	HEFT (8.4)	HEFT (12.4)	HEFT (8.4)	HEFT (12.4)
	10 tasks	HEFT (13.3)	CPOP (36.3)	CPOP (13.7)	HEFT (25.0)
	20 tasks	CPOP (16.3)	CPOP (69.3)	CPOP (20.0)	HEFT (55.3)
4×4	5 tasks	HEFT (8.2)	RR (54.0)	HEFT (8.9)	HEFT (8.9)
	10 tasks	HEFT (16.4)	RR (253.9)	HEFT (15.0)	RR (184.8)
	20 tasks	HEFT (22.2)	RR (937.4)	HEFT (15.9)	HEFT (676.2)

TABLE VII: Interference impact: makespan (seconds) without interference vs. with csma_bianchi across all schedulers and routing strategies. Slowdown is the relative increase. Entries $>500\%$ highlighted.

Network	DAG	Widest-path			Shortest-path		
		HEFT	CPOP	RR	HEFT	CPOP	RR
2×2	5t	0%	0%	+51%	0%	0%	+24%
	10t	+11%	+54%	+66%	+44%	+39%	+38%
	20t	+80%	+67%	+108%	+31%	+31%	+55%
3×3	5t	+47%	+47%	+74%	+47%	+47%	+74%
	10t	+282%	+166%	+1440%	+82%	+111%	+437%
	20t	+441%	+325%	+2446%	+176%	+181%	+688%
4×4	5t	+3735%	+3735%	+219%	0%	0%	+143%
	10t	+8430%	+7895%	+799%	+2790%	+2809%	+612%
	20t	+6768%	+6995%	+2490%	+4161%	+3490%	+1811%

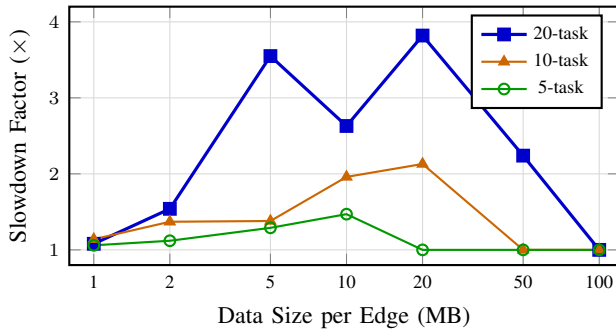


Fig. 6: Interference slowdown vs. data size per edge (3×3 grid, HEFT, shortest-path). Slowdown peaks at intermediate CCR values (≈ 0.3 – 1.1) where transfers are long enough to contend but not so large that the scheduler collapses tasks onto one node.

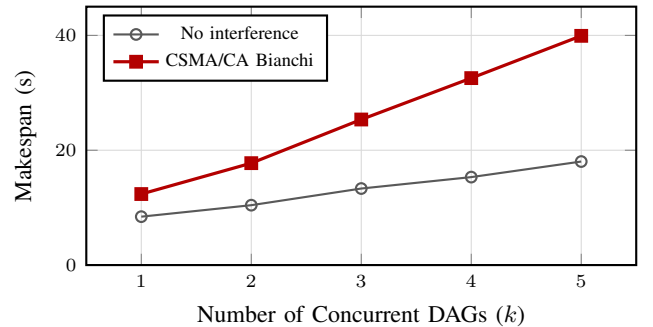


Fig. 7: Multi-DAG makespan scaling on 3×3 grid (5-task DAGs, HEFT, shortest-path). Contention amplifies super-linearly: the gap between the curves widens from $1.47 \times$ ($k=1$) to $2.21 \times$ ($k=5$), indicating positive feedback in contention.

creates a positive feedback loop—each additional DAG’s transfers contend with transfers from all previously injected DAGs, amplifying the per-DAG overhead.

VII. DISCUSSION AND LIMITATIONS

A. When is Flow-Level Modeling Trustworthy?

`ncsim`’s flow-level approach (Table II) is appropriate for the bulk-transfer workloads studied here (10 MB payloads per edge), where steady-state throughput dominates and transient effects like TCP slow-start contribute negligibly to total transfer

time. For workloads with many short transfers (e.g., sub-100 KB control messages), the assumption of instantaneous convergence to Bianchi’s saturation throughput may not hold, and packet-level simulation would be warranted.

The contention model assumes saturation conditions (all contending stations always have packets to send), which over-approximates interference during periods of low network load. This makes our interference-aware makespans conservative upper bounds, meaning the true rank inversion rate may be lower but the qualitative phenomenon persists.

B. Limitations of the Regret Analysis

Our study covers three schedulers, three DAG sizes, and three network topologies, all with deterministic parameters (fixed seed, no stochastic fading). While sufficient to demonstrate the existence and magnitude of rank inversions, several limitations apply:

- **Scheduler coverage.** More sophisticated interference-aware schedulers (e.g., contention-zone-aware HEFT variants) might avoid rank inversions entirely. Our results motivate the development of such schedulers.
- **Deterministic scenarios.** Real wireless channels exhibit fading and mobility. Stochastic effects might smooth or amplify rank inversions.
- **Single-slot compute.** Each node executes one task at a time. Multi-slot parallelism (e.g., GPU inference) would change the compute-communication balance.
- **Network scale.** Our largest topology is 4×4 (16 nodes). Scaling to 50–100 node networks is computationally feasible with `ncsim` but was not explored here.

VIII. CONCLUSION

We have presented empirical evidence that wireless interference causes *scheduler rank inversions*: the scheduler that appears optimal under an interference-free model is dominated under realistic CSMA/CA conditions. In our 108-run study, this occurs in 38.9% of scenarios, with relative regret reaching 484%—meaning a practitioner who trusts an interference-free model deploys a scheduler that is nearly $6\times$ slower than the best alternative. The most striking finding is that the simplest scheduler (round-robin) outperforms HEFT in large wireless meshes because HEFT’s parallelism-maximizing placements create dense contention that overwhelms the scheduling gains.

These results carry a practical implication: **DAG scheduling research for wireless edge computing must model wireless interference, not just bandwidth.** Evaluations that assume uncontested links risk not only inaccurate makespan predictions but systematically wrong conclusions about which algorithms are superior.

The analysis was enabled by `ncsim`, a lightweight simulator whose Bianchi-based CSMA/CA model matches published analytical results within 0.003%. Future work includes developing interference-aware scheduling heuristics that account for contention during task placement, extending the analysis to dynamic rescheduling triggered by observed contention, incorporating node mobility with time-varying topology, and validating rank inversion phenomena against ns-3 packet-level simulations at larger scales.

The `ncsim` simulator and all experimental configurations are available as open-source software to support reproducibility and further investigation.

REFERENCES

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [2] ns-3 Consortium, “ns-3: A Discrete-Event Network Simulator,” <https://www.nsnam.org/>, accessed Mar. 2026.
- [3] A. Varga and R. Hornig, “An Overview of the OMNeT++ Simulation Environment,” in *Proc. ICST SIMUTools*, 2008.
- [4] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, “Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms,” *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2899–2917, 2014.
- [5] H. Casanova, S. Pandey, J. Oeth, R. Tanaka, F. Suter, and R. Ferreira da Silva, “WRENCH: A Framework for Simulating Workflow Management Systems,” in *Proc. WORKS Workshop*, 2018, pp. 74–85.
- [6] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, “CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms,” *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.
- [7] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, “iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in the Internet of Things, Edge and Fog Computing Environments,” *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.
- [8] C. Sonmez, A. Ozgovde, and C. Ersoy, “EdgeCloudSim: An Environment for Performance Evaluation of Edge Computing Systems,” *Transactions on Emerging Telecommunications Technologies*, vol. 29, no. 11, e3493, 2018.
- [9] G. Bianchi, “Performance Analysis of the IEEE 802.11 Distributed Coordination Function,” *IEEE Journal on Selected Areas in Communications*, vol. 18, no. 3, pp. 535–547, Mar. 2000.
- [10] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.